

AI Data Analyst Chatbot

AS-IS → TO-BE Technical Roadmap, Build Plan and Objective Checklist

Python • LangChain • Ollama • Groq • Pydantic • SQLite • RAG • Chroma • Pandas • SQL

Purpose: provide a practical, version-by-version implementation plan that can be followed while coding. Each version has a defined outcome, tasks, implementation guidance, validation criteria, and a checklist.

How to Use This Document

- ☐ Build one version at a time. Do not start the next version until the current version passes its validation criteria.
- ☐ Commit to Git after every working objective or completed version.
- ☐ Keep the existing application working while refactoring. Avoid a large rewrite unless a later version explicitly requires it.
- ☐ Use the checklist boxes as progress gates. Tick an item only after you have run and verified it.
- ☐ When an error occurs, capture the full traceback before changing several things at once.
- ☐ Keep secrets out of Git. The .env file and local database files should remain ignored.

1. Executive Summary

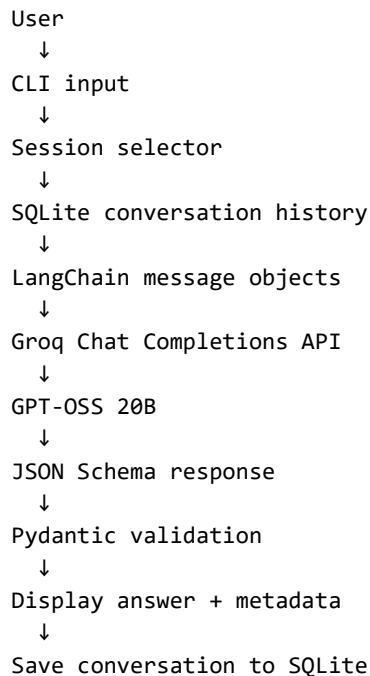
The project will evolve from a command-line conversational chatbot into a modular AI Data Analyst. The final system will be able to maintain conversation context, use local or cloud LLM inference, retrieve information from documents through RAG, inspect and analyse structured datasets, query databases, generate visualisations, and return predictable structured results.

Stage	Primary capability	Core technologies
V1	Existing conversational chatbot	Groq, LangChain, SQLite, Pydantic
V2	Local/cloud model switching	Ollama, Groq, LangChain
V3	Document RAG	Embeddings, Chroma, retriever
V4	Data ingestion and profiling	Pandas, CSV, Excel
V5	Data-analysis agent	LangChain tools, Python/Pandas
V6	SQL analysis	SQLAlchemy, SQL Server/SQLite
V7	Charts and analytical reporting	Pandas, Matplotlib
V8	Integrated AI Data Analyst	Agent + RAG + data tools + memory
V9	Application/API layer	FastAPI or ASP.NET Core + web UI
V10	Production hardening	Testing, logging, security, deployment

2. AS-IS: Current Application

The current application is a command-line Python chatbot with persistent conversational memory and structured AI responses.

Current flow



Current capabilities

- ✓ Interactive command-line chat.
- ✓ Persistent SQLite conversation history.
- ✓ Multiple conversation sessions.
- ✓ Commands for /history, /clear and /exit.
- ✓ Groq cloud inference using openai/gpt-oss-20b.
- ✓ Pydantic validation of the response.
- ✓ Structured fields: answer, topic, difficulty and confidence.
- ✓ Environment-variable based API key loading.
- ✓ Git-ready project with a .gitignore.

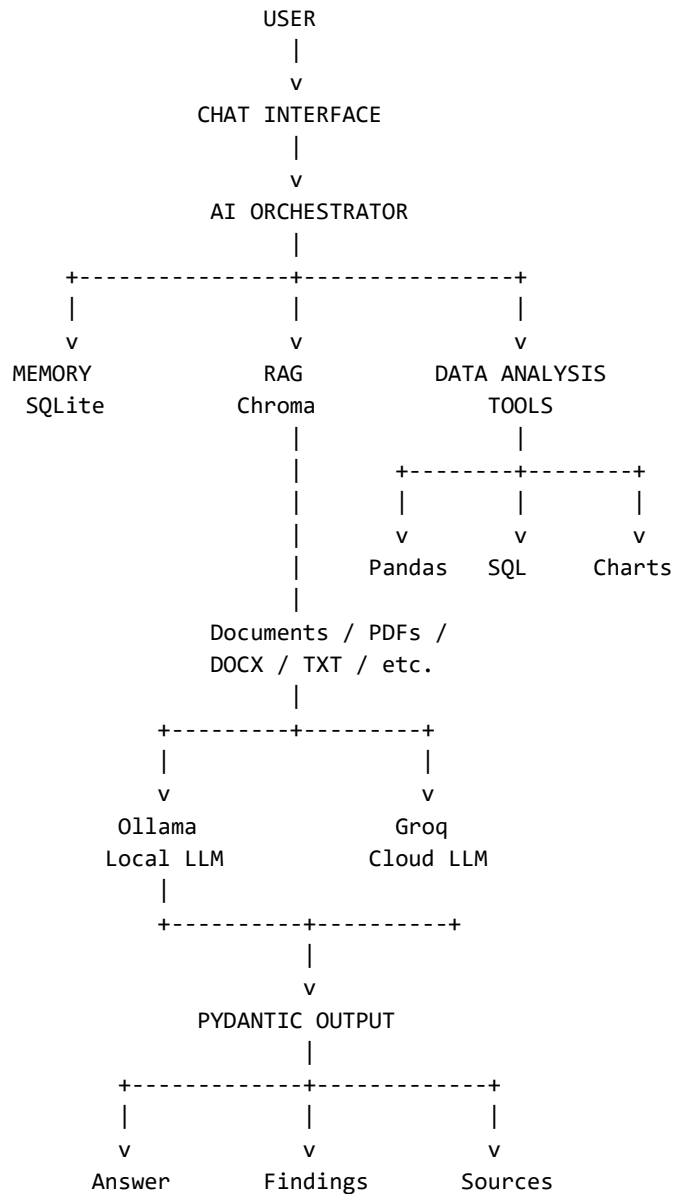
Current limitations

- The application depends on Groq for inference.
- There is no local LLM provider.
- There is no document ingestion or semantic retrieval.
- There is no vector database.
- The model cannot reliably perform arbitrary dataset calculations by itself.
- There is no dedicated CSV/Excel data-analysis layer.
- There is no SQL analysis tool.
- There is no chart-generation workflow.
- The application is still largely contained in one main program rather than modular components.

- The CLI is the primary interface.

3. TO-BE: Final Application

The target system is a modular AI Data Analyst that combines an LLM with deterministic software tools. The LLM decides what should happen; Python, Pandas, SQL and retrieval systems perform the operations.



4. Target Repository Structure

```

ai-agent/
|
|-- main.py
|-- config.py
|-- requirements.txt
|-- .env
|-- .gitignore

```

```
├── README.md
├── models/
│   └── response_models.py
├── llm/
│   ├── groq_provider.py
│   ├── ollama_provider.py
│   └── router.py
├── memory/
│   └── sqlite_memory.py
├── rag/
│   ├── document_loader.py
│   ├── embeddings.py
│   ├── vector_store.py
│   └── retriever.py
├── data_analysis/
│   ├── loader.py
│   ├── profiler.py
│   ├── analyzer.py
│   ├── sql_tools.py
│   └── visualizer.py
├── agents/
│   └── data_agent.py
├── data/
├── documents/
├── outputs/
└── tests/
```

5. Development Rules

- ☐ Use Python 3.13 initially, but verify package compatibility before adding a dependency.
- ☐ Keep API keys in .env and never commit them.
- ☐ Use Git commits as checkpoints: one commit per meaningful working milestone.
- ☐ Prefer deterministic Python/Pandas/SQL calculations over asking the LLM to calculate raw numbers.
- ☐ Use RAG for unstructured knowledge, not as a replacement for database queries or dataframe operations.
- ☐ Validate LLM-generated structured output with Pydantic.
- ☐ Keep model providers behind a common interface so Groq and Ollama can be swapped.
- ☐ Start with small datasets and a small document collection before scaling.
- ☐ Do not give the model unrestricted operating-system access. Tools should expose only controlled operations.

VERSION 1 — Stabilise the Existing Chatbot

Goal: create a clean, reproducible baseline before introducing Ollama, RAG or data analysis.

V1 target

CLI → Session Management → SQLite Memory → Groq → Structured JSON → Pydantic → Display

Tasks

- √ Confirm the application runs from a clean terminal.
- √ Confirm .env contains GROQ_API_KEY and is ignored by Git.
- √ Confirm chat_history.db is ignored by Git.
- √ Confirm session creation and selection work.
- √ Confirm /history, /clear and /exit work.
- √ Confirm structured responses contain answer, topic, difficulty and confidence.
- √ Confirm Pydantic rejects invalid response data.
- √ Create a Git commit called something like: V1 - stable chatbot baseline.

How to validate

- √ Ask a normal question and verify a response is returned.
- √ Exit and reopen the same session; verify old messages load.
- √ Create a second session; verify it does not contain the first session's messages.
- √ Run /clear and verify the selected session has no history.
- √ Inspect the SQLite database and verify message persistence.
- √ Check Git status and verify .env is not tracked.

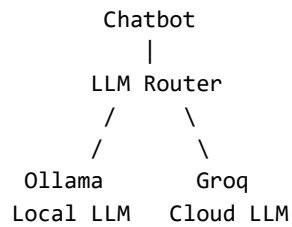
V1 completion gate

Do not move forward until the existing application is reliable and reproducible.

VERSION 2 — Add Ollama and Model Switching

Goal: allow the application to use a local Ollama model or Groq without changing the rest of the chatbot.

Architecture change



Step 1 — Install Ollama

Install Ollama for Windows from the official Ollama website. After installation, verify the CLI.

```
ollama --version
```

Step 2 — Pull a model

Start with a model that your hardware can run comfortably. Do not assume a 20B model will be fast on an 8 GB RAM machine.

```
ollama pull gpt-oss:20b
ollama list
```

If the 20B model is too slow or memory-heavy, test a smaller suitable model instead.

Step 3 — Install the LangChain integration

```
pip install -U langchain-ollama
```

Step 4 — Create an Ollama provider

```
from langchain_ollama import ChatOllama

llm = ChatOllama(
    model="gpt-oss:20b",
    temperature=0
)
```

Step 5 — Create a common provider interface

The rest of the application should not care whether the LLM is Groq or Ollama. Put provider-specific code in separate modules.

```
def get_llm(provider: str):
    if provider == "ollama":
        return create_ollama_llm()

    if provider == "groq":
        return create_groq_llm()

    raise ValueError("Unsupported provider")
```

Step 6 — Add configuration

```
LLM_PROVIDER=ollama
```

Use configuration rather than editing application logic every time you switch providers.

V2 checklist

- ☐ Ollama is installed and responds to `ollama --version`.
- ☐ At least one local model is downloaded.
- ☐ A simple prompt works against Ollama.
- ☐ Groq still works.
- ☐ Provider selection works.
- ☐ The chatbot can switch providers without changing the conversation/session layer.
- ☐ A failed local-model call produces a readable error.
- ☐ A Git commit is created.

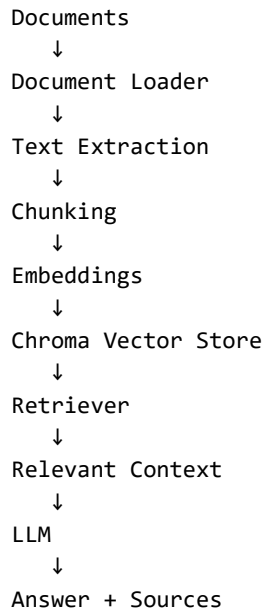
V2 completion gate

You should now be able to run the same chatbot with either a local or cloud model.

VERSION 3 — Build the RAG System

Goal: let the chatbot answer questions using your own documents.

RAG pipeline



Step 1 — Install packages

```
pip install -U chromadb langchain-chroma langchain-ollama
```

Add an appropriate document loader package when you choose your first document formats.

Step 2 — Create document folders

```
documents/  
├── pdf/  
├── word/  
└── text/
```

Step 3 — Load documents

Create `rag/document_loader.py`. Start with TXT and PDF. Add DOCX after the basic pipeline works.

Step 4 — Split documents

Use a recursive text splitter. Begin with moderate chunk sizes and overlap, then tune based on retrieval quality.

Step 5 — Generate embeddings

Use a local Ollama embedding model where practical. Keep the embedding model fixed after indexing; changing it requires re-indexing.

Step 6 — Store embeddings in Chroma

Persist the vector database locally so you do not re-embed every document every time the application starts.

Step 7 — Build the retriever

Create a function that accepts a user question and returns the most relevant document chunks plus metadata such as filename and page.

Step 8 — Connect retrieval to the chatbot

The application should retrieve context before generating an answer. The prompt should instruct the model to distinguish retrieved evidence from unsupported claims.

RAG test questions

- ☐ Ask a question whose answer is explicitly present in a document.
- ☐ Ask a question whose answer is not present and verify the model does not confidently invent a document-based answer.
- ☐ Ask a question requiring two different document sections.
- ☐ Verify source filenames/page metadata are retained.
- ☐ Change a document, re-index it, and verify retrieval reflects the change.

V3 checklist

- ☐ Documents can be loaded.
- ☐ Documents are chunked.
- ☐ Embeddings are generated.
- ☐ Chroma is persisted.
- ☐ Retriever returns relevant chunks.
- ☐ Retrieved context is supplied to the LLM.
- ☐ Answers can identify their source documents.
- ☐ A Git commit is created.

VERSION 4 — Data Ingestion and Dataset Profiling

Goal: make Python understand structured datasets before asking an LLM to analyse them.

Supported initial formats

- ☐ CSV
- ☐ Excel (.xlsx)
- ☐ Data already stored in Pandas DataFrames

Create data_analysis/loader.py

```
import pandas as pd

def load_csv(path):
    return pd.read_csv(path)

def load_excel(path, sheet_name=0):
    return pd.read_excel(path, sheet_name=sheet_name)
```

Create a dataset profiler

The profiler should produce deterministic metadata rather than asking the LLM to inspect every row.

```
def profile_dataframe(df):
    return {
        "rows": len(df),
        "columns": list(df.columns),
        "dtypes": df.dtypes.astype(str).to_dict(),
        "missing_values": df.isna().sum().to_dict(),
        "numeric_summary": df.describe().to_dict()
    }
```

V4 objectives

- ☐ Load CSV files.
- ☐ Load Excel files.
- ☐ Report row and column counts.
- ☐ Report column names and data types.
- ☐ Report missing values.
- ☐ Report basic descriptive statistics.
- ☐ Detect likely date columns.
- ☐ Detect numeric and categorical columns.
- ☐ Handle invalid file paths and malformed files safely.
- ☐ Create sample datasets for testing.

V4 completion gate

Before using an LLM, your Python layer must reliably load and describe a dataset.

VERSION 5 — Build the Data-Analysis Agent

Goal: allow the LLM to choose controlled analytical tools while Python performs the actual calculations.

Core principle

Never rely on the LLM to perform large numerical calculations from raw data. The LLM should formulate an operation; Pandas should execute it; the result should be returned to the LLM for interpretation.

Example flow

User:
"What was average laptop revenue in Gauteng?"

LLM / Agent:
Choose dataframe analysis tool

Python:
Filter Product == Laptop
Filter Region == Gauteng
Calculate mean Revenue

Tool result:
R 140,250

LLM:
Explain result to user

Initial tools

- ☐ Implement inspect_dataset
- ☐ Implement filter_dataset
- ☐ Implement aggregate_dataset
- ☐ Implement calculate_statistics
- ☐ Implement group_by_column
- ☐ Implement sort_dataset
- ☐ Implement get_unique_values

Tool design rule

Each tool should have a narrow purpose, validated inputs, predictable output and clear errors. Avoid exposing unrestricted eval/exec as the first implementation.

Pydantic tool/input models

```
class GroupByRequest(BaseModel):  
    column: str  
    metric: str  
    operation: str
```

V5 test questions

- ☐ How many rows are in the dataset?
- ☐ What are the top five products by revenue?
- ☐ What is the average revenue?

- ☐ Which region generated the most revenue?
- ☐ What is monthly revenue?
- ☐ What percentage of rows contain missing values?

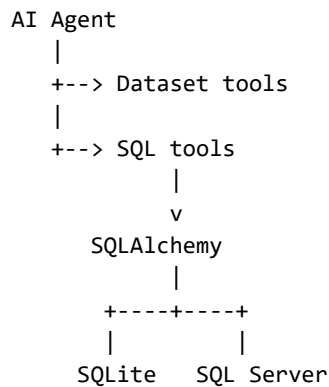
V5 completion gate

The system must calculate answers through deterministic tools and then explain the results.

VERSION 6 — Add SQL Database Analysis

Goal: extend analysis beyond files to relational databases.

Target architecture



Implementation sequence

- ☐ Start with SQLite because it is already part of the project.
- ☐ Create a sample analytical database.
- ☐ Use SQLAlchemy for database connectivity.
- ☐ Expose safe read-only query capabilities first.
- ☐ Add schema inspection.
- ☐ Add table/column discovery.
- ☐ Only allow SELECT-style analytical operations initially.
- ☐ Add SQL query validation and limits before considering broader database access.
- ☐ Test against SQL Server after SQLite is stable.

Security rule

The first SQL version should be read-only. Do not let an LLM freely execute INSERT, UPDATE, DELETE, DROP or ALTER statements.

V6 completion gate

The chatbot can answer analytical questions using both files and a relational database.

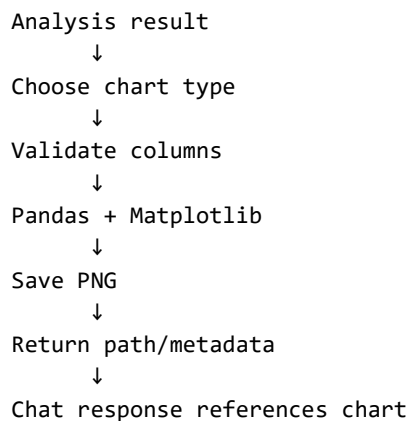
VERSION 7 — Visualisation and Analytical Reports

Goal: turn numerical results into charts and concise analytical findings.

Initial visualisations

- ☐ Line chart for time-series trends.
- ☐ Bar chart for category comparisons.
- ☐ Histogram for distributions.
- ☐ Scatter plot for relationships between numeric variables.
- ☐ Box plot for distributions/outliers where appropriate.

Visualisation flow



Reporting output

A report should distinguish facts, calculated findings, interpretations and recommendations. For example: 'Revenue decreased 12%' is a calculated finding; 'the decline appears concentrated in Product X' is an interpretation.

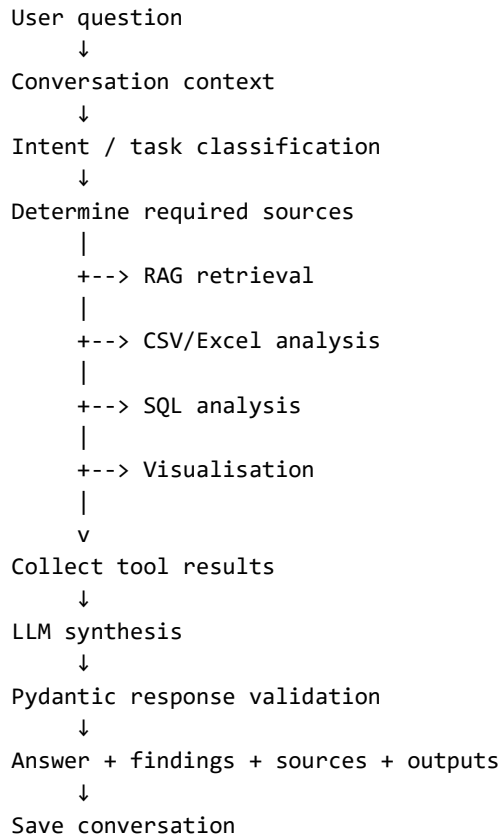
V7 checklist

- ☐ Charts can be generated from DataFrames.
- ☐ Charts are saved to outputs/.
- ☐ Invalid column selections are handled.
- ☐ Charts have useful titles and axis labels.
- ☐ The agent can select an appropriate chart for a question.
- ☐ The final response identifies the generated visualisation.

VERSION 8 — Integrate Everything into the AI Data Analyst

Goal: combine memory, RAG, LLM providers, data tools, SQL and visualisation behind one orchestrator.

Final reasoning flow



Suggested response model

```
class AnalysisResponse(BaseModel):
    answer: str
    analysis_type: str
    datasets_used: list[str]
    calculations_performed: list[str]
    key_findings: list[str]
    sources: list[str]
    confidence: float
```

Important behaviour

- ☐ Use RAG when the question requires unstructured documents.
- ☐ Use Pandas when the question requires file-based calculations.
- ☐ Use SQL when the required data lives in a relational database.
- ☐ Use more than one source when the question requires contextual and quantitative evidence.
- ☐ Do not fabricate a result when a tool failed.
- ☐ Expose source information where possible.
- ☐ Record which datasets/tools were used.
- ☐ Preserve conversation/session context.

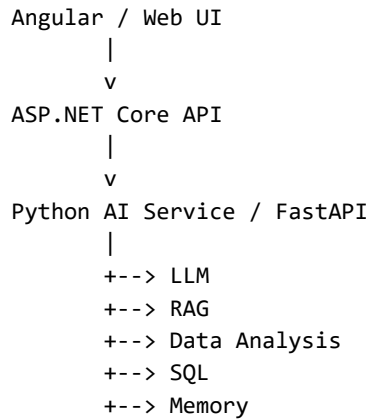
End-to-end acceptance tests

- ☐ Ask a document-only question.
- ☐ Ask a dataset-only question.
- ☐ Ask a SQL-only question.
- ☐ Ask a question requiring both a dataset and a document.
- ☐ Ask for a chart.
- ☐ Ask a follow-up question referring to a previous result.
- ☐ Switch from Ollama to Groq and repeat a question.
- ☐ Start a new session and verify context isolation.

VERSION 9 — Expose the AI Analyst through an API and Web Interface

Goal: move beyond the CLI while keeping the Python AI engine separate from the user interface.

Recommended architecture



Because the existing development stack includes Angular and ASP.NET Core, a strong portfolio architecture is to keep the web/API layer separate from the Python AI/data-analysis service.

API endpoints to introduce

- ☐ Design and implement POST /chat
- ☐ Design and implement POST /sessions
- ☐ Design and implement GET /sessions
- ☐ Design and implement DELETE /sessions/{id}
- ☐ Design and implement POST /documents
- ☐ Design and implement POST /datasets
- ☐ Design and implement GET /datasets
- ☐ Design and implement POST /analyse
- ☐ Design and implement GET /reports/{id}

V9 completion gate

The AI engine can be called through an API and the CLI becomes only one possible client.

VERSION 10 — Production Hardening

Goal: make the system safer, testable, observable and deployable.

- ☐ Move configuration into environment variables/settings.
- ☐ Add unit tests for loaders, profilers, tools and Pydantic models.
- ☐ Add integration tests for LLM providers.
- ☐ Add RAG retrieval tests.
- ☐ Add data-analysis regression tests using known datasets.
- ☐ Add structured application logging.
- ☐ Add request/session identifiers.
- ☐ Add file size and type limits.
- ☐ Restrict SQL operations to safe read-only analysis.
- ☐ Sandbox or tightly constrain code execution if introduced.
- ☐ Validate all tool inputs.
- ☐ Add timeouts and error handling.
- ☐ Add rate limiting where appropriate.
- ☐ Document data privacy and retention behaviour.
- ☐ Create deployment configuration.
- ☐ Create a professional README and architecture diagram.

6. Recommended Data-Analysis Methodology

For every analytical question, the system should follow a repeatable workflow.

1. Understand the question and identify the requested metric or decision.
2. Identify the required dataset(s) or document(s).
3. Inspect schema, columns, data types and missing values.
4. Select an appropriate deterministic operation.
5. Execute the operation using Pandas or SQL.
6. Validate the result.
7. Generate a visualisation if it improves interpretation.
8. Use RAG to add business/document context when required.
9. Ask the LLM to synthesise the evidence.
10. Return findings, methodology, sources and limitations.

7. RAG vs Data Analysis: Decision Guide

Question type	Primary system	Why
What does the policy say?	RAG	Information is in documents.
What was total revenue?	Pandas/SQL	Requires exact calculation.
Which product grew fastest?	Pandas/SQL	Requires aggregation/comparison.
Why did revenue decline?	Pandas/SQL + RAG	Quantitative evidence plus contextual documents.
What does the annual report say	RAG	Document retrieval.

about revenue?		
Show monthly sales trend	Pandas/SQL + visualisation	Time-series calculation and chart.
Compare performance with strategy	Pandas/SQL + RAG	Data plus strategic context.

8. Example Final User Experience

Example question:

“Analyse our 2026 sales performance and explain the main reasons for the decline in Q3.”

Expected internal workflow:

11. Load/inspect sales dataset.
12. Calculate quarterly revenue.
13. Compare Q3 with Q1/Q2 and prior period if available.
14. Break the decline down by product, region and other relevant dimensions.
15. Identify the largest contributors to the decline.
16. Retrieve relevant passages from reports or strategy documents.
17. Compare quantitative findings with documented business explanations.
18. Generate a trend chart and contribution chart.
19. Produce a structured analytical response.
20. State limitations where data is insufficient.

9. Suggested Git Milestones

Commit	Milestone
V1	Stable chatbot baseline
V2	Ollama and provider switching
V3	RAG pipeline
V4	Data ingestion and profiling
V5	Data-analysis tools/agent
V6	SQL analysis
V7	Visualisation and reports
V8	Integrated AI Data Analyst
V9	API/web integration
V10	Production hardening

10. Overall Master Checklist

- ☐ V1 — Stable existing chatbot
- ☐ V2 — Ollama local inference
- ☐ V2 — Groq/Ollama switching
- ☐ V3 — Document ingestion
- ☐ V3 — Embeddings
- ☐ V3 — Chroma vector store
- ☐ V3 — Retrieval

- ☐ V4 — CSV ingestion
- ☐ V4 — Excel ingestion
- ☐ V4 — Dataset profiling
- ☐ V5 — Data-analysis tools
- ☐ V5 — Agent tool selection
- ☐ V6 — SQL connectivity
- ☐ V6 — Safe read-only SQL analysis
- ☐ V7 — Charts
- ☐ V7 — Analytical reports
- ☐ V8 — Integrated RAG + data analysis
- ☐ V8 — Structured analytical responses
- ☐ V9 — Python API
- ☐ V9 — Web interface
- ☐ V10 — Automated tests
- ☐ V10 — Logging and security
- ☐ V10 — Deployment
- ☐ V10 — Professional documentation

11. Recommended Build Order for Coding Alongside This Document

Do not attempt to build the entire system at once. The safest progression is:

21. First, make V1 completely stable.
22. Second, add Ollama without removing Groq.
23. Third, build RAG independently and test retrieval before integrating it deeply.
24. Fourth, build the Pandas data-analysis engine without an LLM.
25. Fifth, expose the Pandas operations as controlled agent tools.
26. Sixth, add SQL as another controlled toolset.
27. Seventh, add visualisation.
28. Eighth, integrate all capabilities behind an orchestrator.
29. Ninth, expose the Python service through an API.
30. Tenth, harden, test and document the system.

12. Definition of Done for the Final Project

- ☐ A user can start a conversation and select a session.
- ☐ The system can use Ollama locally and Groq remotely.
- ☐ The system maintains persistent conversation memory.
- ☐ Users can provide documents for retrieval.
- ☐ The system can retrieve relevant document evidence.
- ☐ Users can analyse CSV and Excel datasets.
- ☐ The system can query approved relational data sources.
- ☐ Numerical results are calculated by deterministic tools.

- ☐ The system can generate useful charts.
- ☐ The LLM can combine quantitative results with document context.
- ☐ Responses are validated with Pydantic.
- ☐ The application reports sources and important limitations.
- ☐ Secrets and private data are not committed to Git.
- ☐ Automated tests cover critical components.
- ☐ The repository has clear setup instructions and architecture documentation.

13. First Coding Session — Exact Starting Point

When you are ready to begin, do not start by adding RAG. Start with V1 and then implement V2. The first practical sequence is:

- ☐ Create a Git branch/checkpoint for the current working application.
- ☐ Verify the current chatbot works.
- ☐ Verify .gitignore protects .env and local databases.
- ☐ Install Ollama.
- ☐ Run `ollama --version`.
- ☐ Pull a suitable local model.
- ☐ Install `langchain-ollama`.
- ☐ Create `llm/ollama_provider.py`.
- ☐ Create `llm/groq_provider.py`.
- ☐ Create `llm/router.py`.
- ☐ Add `LLM_PROVIDER` configuration.
- ☐ Run the same test question through Ollama and Groq.
- ☐ Commit the working V2.

END OF ROADMAP